SIMATS ENGINEERING

CRYPTOGRAPHY AND NETWORK SECURITY (CSA51)
ASSESSMENT TOOL 3 — DEBUGGING & OPTIMISATION

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange to strengthen information security.

Note: The following are model answers for all five questions in the supplied assessment. They explain the debugging approach, optimisations, and security/performance trade-offs.

1. Debugging and Optimization of Symmetric Encryption (DES / 3DES)

A. Errors and fixes

Identical plaintext and key should produce identical ciphertext when the algorithm, key, mode, IV/nonce, and padding are identical. In DES, check that the 64-bit key is processed correctly: PC-1 removes parity bits, the 28-bit halves are rotated according to the standard schedule, and PC-2 selects the 48-bit round keys. Verify the initial permutation, 16 Feistel rounds, expansion, S-box substitutions, P permutation, final swap, and inverse final permutation. Common errors include incorrect bit numbering (DES tables use 1-based positions), wrong rotation counts, using the wrong bit order, omitting a permutation, and mutating shared state between runs. Use fixed known-answer test vectors and compare intermediate round keys and round outputs against a trusted implementation.

B. Performance improvements

Process data in chunks rather than loading a file of 10 MB or more into memory at once. Avoid repeated list/string conversions and unnecessary bit-by-bit Python loops; use byte-oriented operations, precomputed tables where appropriate, and a well-tested cryptographic library with native implementations. Reuse cipher objects only where the library permits and where mode/nonce rules remain correct. Benchmark with the same dataset, hardware, and settings, and verify the output against test vectors after each change. DES has small blocks and is slow in pure Python; 3DES performs three DES operations per block and is generally slower.

C. Modern cipher comparison

Yes. Replacing DES/3DES with AES generally improves security and usually improves performance on modern systems, especially when hardware acceleration is available. DES has only a 56-bit effective key and is insecure against brute force. 3DES has legacy compatibility value but is slow and is being phased out. AES supports 128-, 192-, and 256-bit keys and is widely supported. For new systems, use a vetted authenticated-encryption mode such as AES-GCM (or ChaCha20-Poly1305 where appropriate), with a unique nonce for every encryption under the same key. Do not design a cipher yourself.

2. Performance Analysis of Block Cipher Modes of Operation

A. Padding and IV handling

ECB does not use an IV and reveals repeated plaintext blocks, so it should not be used for ordinary data protection. CBC requires a fresh, unpredictable IV for each encryption and correct block padding when the plaintext is not block-aligned; remove padding only after successful validation. CFB and OFB are stream-like modes and do not require PKCS#7 block padding. Decryption failures often result from using a different IV, truncating or misplacing the IV, padding twice, unpadding incorrectly, or passing text instead of bytes. Store/transmit the IV with the ciphertext, but never reuse mode-specific nonces/IVs in ways prohibited by the mode.

B. Reduce time and memory use

Use bytes and memory views instead of repeatedly converting between strings, lists, and byte arrays. Process large files in bounded chunks where the library supports streaming. Create the cipher context once per message, avoid recalculating keys or round schedules for every block, and avoid concatenating immutable byte strings inside loops; append to a bytearray or write output incrementally. Profile before optimising and test empty, short, exact-block, and multi-block messages.

C. Mode comparison and recommendation

ECB may look fast but is insecure because identical plaintext blocks produce identical ciphertext blocks. CBC needs sequential block processing for encryption and padding; its encryption is not naturally parallel. CFB and OFB can process data as a stream, but do not provide integrity by themselves. For modern real-time communication, prefer an

authenticated-encryption mode such as AES-GCM, which provides confidentiality and integrity and can be efficient with hardware support. If the question is limited to ECB/CBC/CFB/OFB, performance depends on implementation and hardware; there is no universally fastest mode. Select based on security requirements, benchmark fairly, and authenticate data.

3. Debugging RSA Cryptosystem Implementation

A. Likely causes and corrections

RSA works on integers, not arbitrary-length text strings directly. A large message may exceed the maximum plaintext size for one RSA operation, causing errors or incorrect results. Use RSA-OAEP to encrypt a short random symmetric key, then encrypt the message with an authenticated symmetric cipher (hybrid encryption). Generate distinct large random primes p and q using a cryptographic random source; compute n = p × q and $\phi(n) = (p–1)(q–1)$, choose e such that gcd(e, ϕ(n)) = 1, and calculate d as the modular inverse of e modulo ϕ(n). Ensure the same encoding, padding scheme, key, and byte-to-integer conversion are used during encryption and decryption. Never use textbook RSA without secure padding.

B. Optimisation techniques

Use square-and-multiply (binary modular exponentiation), which computes a^b mod n in O(log b) modular multiplications rather than multiplying a repeatedly. During private-key decryption, the Chinese Remainder Theorem (CRT) can accelerate computation by calculating results modulo p and q and recombining them. Use a maintained cryptographic library because it implements constant-time operations and safer key handling more reliably than handwritten Python code. Validate that decrypted data passes the expected decoding and authentication checks.

C. Key size, security, and speed

Larger RSA keys generally increase security but require more computation and storage. A 2048-bit RSA key is a common minimum for many current applications; 3072-bit RSA is often selected for a higher security margin. Exact requirements depend on policy and expected lifetime. Larger keys slow key generation and private-key operations. RSA is therefore best used to protect small values such as symmetric keys, not large files. Hybrid encryption combines RSA-OAEP for key transport with AES-GCM for bulk data.

4. Optimization and Security Analysis of Diffie–Hellman Key Exchange

A. Shared-key debugging

Let p be a suitable large prime and g a valid generator or subgroup element. Alice chooses secret a and sends A = g^a mod p; Bob chooses secret b and sends B = g^b mod p. Alice calculates K = B^a mod p, while Bob calculates K = A^b mod p. Both obtain g^(ab) mod p. Incorrect keys commonly come from using ordinary exponentiation without the modulus, applying the modulus at the wrong stage, mixing up public and private values, using different p or g values, or converting values inconsistently. Check that received public values are in the permitted range and belong to the intended group.

B. Performance improvements

Use built-in modular exponentiation, pow(g, a, p), rather than computing g**a and reducing afterward; the built-in form performs modular exponentiation efficiently and avoids creating an enormous intermediate integer. Use well-established, appropriately sized groups and cryptographic libraries. Cache only public, reusable group parameters when safe; do not reuse ephemeral private secrets where fresh ephemeral keys are required. Measure key-generation and exchange time separately.

C. Preventing man-in-the-middle attacks

Unauthenticated Diffie–Hellman alone does not prove who is at the other end, so an attacker can establish separate keys with both parties. Authenticate the exchange using digital signatures and certificates, or use an established protocol such as TLS with authenticated key exchange. Confirm the peer's identity, validate certificates, and bind the exchanged public values into the authenticated transcript. Authentication adds computation and message-validation overhead, but it prevents key substitution and is worth the cost. Use a reviewed protocol rather than inventing one.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC)

A. Point addition and scalar multiplication

ECC points must be handled according to the selected curve equation and field. For a short Weierstrass curve $y^2 = x^3 + ax + b$ over a prime field, point addition must handle distinct points, point doubling, the point at infinity (identity), and inverse points where x coordinates match and y coordinates sum to zero modulo p. Every subtraction and division must be performed modulo p; division means multiplication by a modular inverse, not ordinary floating-point division. Validate that input points lie on the curve and are in the correct subgroup. Frequent bugs include missing modulo reductions, incorrect slope formulas, failure to handle special cases, and incorrect scalar bit processing.

B. Optimisation

Use double-and-add scalar multiplication: scan the bits of the scalar, repeatedly double the current point and add when the relevant bit is set. This reduces the number of point operations from a naive repeated-addition approach to O(log k) operations for scalar k. More advanced windowing and projective-coordinate methods can reduce operations further, but should be implemented only with careful review. For real applications, use a trusted ECC library with side-channel protections rather than custom arithmetic.

C. ECC versus RSA

ECC can provide security comparable to RSA with much smaller keys. For example, a 256-bit elliptic-curve key is commonly compared with roughly a 3072-bit RSA key for a similar classical security level, though exact equivalence depends on the schemes and standards used. Smaller keys reduce storage, bandwidth, and often computation, making ECC attractive for mobile devices, embedded systems, and other resource-constrained environments. Performance depends on the curve, library, operation, and hardware; ECC is not automatically faster for every task. Use standard curves and protocols from reputable libraries, and keep private keys protected.

Conclusion

Correct cryptographic implementations require standard test vectors, careful handling of keys and IVs/nonces, efficient built-in arithmetic, and validation of inputs. For practical systems, use established cryptographic libraries and modern authenticated encryption rather than custom implementations or obsolete algorithms.

Academic integrity: Review and understand these model answers, then ensure your final submission follows your instructor’s rules and reflects your own work.